

Labwork 6.2 - Image Manipulations

General Information and Submission Rules:

- In this labwork, you need to design a machine that does image manipulations.
- By the end of the labwork, you must submit it to Yulearn as **studentID_fullname_labwork6.c**.
- Only use concepts covered in the labs so far.

Question 1 - Display Message (25p)

- Create a new function (**greetings**) that displays a welcome message and a list of all available operations to the user.
- This function will expect a decision from a user so that it will execute related functions based on this choice until the user exits.

```
Welcome to the Number Games Machine! By using this machine you can do:
1. Red Channel Image Filter
2. Relative Features
3. Convert Grayscale
4. Exit
Make your choice:
```

Question 2 - Red Channel Image Filter (30p)

- You will take a 3x3 image as an input and represent its pixels as a sequential order (integer values range from 0 to 255).
- Each pixel will have an *RGB (Red-Green-Blue)* value.
- You want to “mute” the image by halving the intensity of the pixel, but only for the *Red values* and only for the *bottom half of the image*.
- Write a function named (**red_filter**) that takes
 - *pixels*,

- *width*,
- *height*

```
>> Enter the size of the image: 3x3
>> Enter the RGB values of the image:
(254, 163, 163)
(75, 39, 45)
(65, 69, 168)
(37, 73, 17)
(220, 174, 120)
(192, 49, 236)
(71, 83, 139)
(64, 193, 150)
(221, 228, 43)
```

```
Output image:
(254, 163, 163)
(75, 39, 45)
(65, 69, 168)
(37, 73, 17)
(110, 174, 120)
(96, 49, 236)
(35, 83, 139)
(32, 193, 150)
(110, 228, 43)
```

Question 3 - Relative Features (30p)

- Write a function named (**relative_features**) that takes an output image of Question 2 as an input, and selects only maximum pixels by comparing two consecutive pixels and creates a 2x2 image.

```
Generated image:
(254, 163, 163)
(65, 73, 168)
(110, 174, 236)
(35, 193, 150)
```

Question 4 - Convert Grayscale (30p)

- Write a function named (**convert_gray**) that takes an output image of Question 3 as an input.
- It takes the average of each pixel's RGB values and if it is divisible by 9 (do not directly take the modulus with 9) then return 1 else 0.
- To find out if the number is divisible by 9 apply this rule: *A number is divisible by 9 if the sum of its digits is a multiple of 9.*

```
Average of pixel 1: 193  
Average of pixel 2: 102  
Average of pixel 3: 173  
Average of pixel 4: 378
```

```
Generated grayscale image
```

```
0 0  
0 1
```